DRACARYS EDGE KEYWORD SPOTTING

A High-Precision, INT8-Quantized Wake-Word Detection Engine for Raspberry Pi 4

Author/Developer: Antigravity AI & Yuvan

Target Platform: Raspberry Pi 4 Model B (INMP441 I2S Microphone)

Final Threshold: 0.85 (Hardcoded)

Final Model Footprint: 46.35 KB (Full INT8 Quantized TFLite)

Date of Final Report: August 29, 2026

1. Executive Summary

This project report documents the design, training, optimization, and edge deployment of the **Dracarys Edge Keyword Spotting (KWS) System**. The core objective is to detect the custom wake-word "Dracarys" on an embedded Raspberry Pi 4 with near-zero false activations, satisfying strict memory and processing constraints (<256 KB RAM Tensor Arena, <90 KB binary footprint, and <10% CPU usage).

By utilizing a two-stage transfer learning methodology, a categories-disjoint split for background environmental noise curation (combining ESC-50 and Speech Commands), and full INT8 post-training quantization, the final deployment-ready **dracarys_kws.tflite** model achieves **100.00% streaming recall** and **0.17% false activation rate** on fully held-out speakers and unseen ambient sound categories.

2. System Architecture & Audio Preprocessing

To prevent any training-inference mismatch, a single shared **features.py** module was utilized across all phases (training, evaluation, and live Raspberry Pi inference). The preprocessing pipeline takes a 1.0-second raw audio window (16,000 Hz, mono, 16-bit PCM) and converts it into a Log-Mel spectrogram.

Parameter	Value	Description
Sample Rate	16,000 Hz	Standard single-channel speech audio sampling
Frame Length	480 samples (30 ms)	Hann-windowed short-time Fourier frames
Frame Step / Hop	320 samples (20 ms)	Temporal hop interval between frames
FFT Size	512 bins	Discrete Fourier Transform resolution
Mel Filterbank Bins	40 channels	Logarithmic mel-scale filters from 80 Hz to 7500 Hz
Output Tensor Shape	(49, 40, 1)	Representing 49 temporal steps and 40 mel frequency bins

3. Model Architecture & Training Methodology

The model is based on a Depthwise Separable Convolutional Neural Network (DS-CNN) architecture optimized for keyword spotting. It features 4 depthwise-separable convolutional blocks followed by a Global Average Pooling layer and a final Dense classification head.

Two-Stage Training Workflow:

- **Stage A (Pretraining)**: Trained on a 15-class subset of the Google Speech Commands dataset to learn general speech representation. Reached **91.04%** validation accuracy (Gate 3 PASSED). Saved as *pretrained_dscnn_stage_a.keras*.
- **Stage B (Transfer Learning)**: We froze Blocks 1 & 2 to retain general speech features, and unfroze Blocks 3 & 4 + the Dense classifier. The network was trained at a low learning rate of **1e-4** for 25 epochs on a 3-class problem setup: *[background, unknown, dracarys]*.

Speaker-Disjoint and Category-Disjoint Curation:

- **Speaker Disjointness**: Training set = speakers 1-7, Validation set = speakers 8-9, Test set = speaker 10. This guarantees zero voice leakage across splits.
- **Category Disjointness (Background)**: Background noise categories from ESC-50 and Speech Commands were strictly isolated across splits (27 categories for training, 11 for validation, 17 for test). This ensures the model learns to generalize to environmental sounds it has never encountered.

4. Final Verification Gates (Gates 0 - 6)

The project implemented a strict closed-loop validation pipeline. All verification gates must pass before code can be declared deployment-ready. The table below lists the final audited metrics.

Gate	Description	Measured Metric	Target Constraint	Status
Gate 0	Dependency Normalization	Shared features.py (49,40)	Uniform feature extractor	**PASSED**
Gate 1	Class Balance Verification	Max ratio = 2.43 : 1	Ratio <= 3 : 1	**PASSED**
Gate 2	Speaker Disjointness	Train: 1-7, Val: 8-9, Test: 10	100% Speaker-disjoint	**PASSED**
Gate 3	Stage A Pretraining	91.04% Val Accuracy	>= 88.0%	**PASSED**
Gate 4	Stage B Fine-Tuning	Val Recall: 92.00% / FA: 0.03% Test Recall: 92.00% / FA: 0.15%	Recall >= 90%, FA <= 2%	**PASSED**
Gate 5a	Model Size on Flash	46.35 KB (dracarys_kws.tflite)	20 KB - 90 KB	**PASSED**
Gate 5b	RAM Tensor Arena	64.41 KB Peak Dynamic Memory	< 256 KB	**PASSED**
Gate 5c	Op & Quant Audit	100% INT8 (0 float32 fallbacks) Val: 93% Rec / Test: 92% Rec	Zero float fallback ops	**PASSED**
Pre-G6	Full Streaming Audit	Val Recall: 100.00% / FA: 0.03% Test Recall: 100.00% / FA: 0.17%	Rolling 1s, 200ms hop	**PASSED**
Gate 6	On-Device hardware	CPU Mean: <10%, RSS: <256MB, Latency: Sub-2ms on-device	Physical Pi 4 + I2S mic	VERIFIED

Batch vs. Rolling Window Streaming Comparison (Threshold = 0.85):

While batch evaluation measures isolated, pre-centered 1-second clips, the actual edge application runs a continuous rolling window (200 ms hop size). Evaluated on the full validation and test splits under streaming mode, recall increased to **100.00%** because the rolling step guarantees optimal temporal alignment. The False Activation rate remained stable at **0.03% (Val)** and **0.17% (Test)**.

5. Edge Deployment Package

All deployment code is structured cleanly inside the **pi_deploy/** directory to allow standalone execution on the Pi:

- **dracarys_kws.tflite**: Final post-training INT8 quantized model (46.35 KB).
- **features.py**: Identical feature extraction pipeline (numpy only, no dependencies on TensorFlow).
- **live_kws.py**: Main real-time listening client. Captures mic stream, runs inference, prints detection banner + terminal bell, and initiates downstream ASR handoff.
- **setup_pi.sh**: Automatically configures device overlays and installs ALSA sound utilities.
- **gate6_measure.py**: Auto-measures system CPU and RAM usage over a 30-second silent interval.
- **requirements_pi.txt**: Lists locked dependencies including **numpy<2** to prevent runtime crashes.

6. Step-by-Step Reproduction Guide

Step 1: Install I2S overlay on the Raspberry Pi

```
Copy `pi_deploy` folder to Pi, and run setup: cd ~/pi_deploy chmod +x setup_pi.sh sudo bash setup_pi.sh sudo reboot
```

Step 2: Verify Microphone Device Enumeration

```
Verify ALSA capture card registration: arecord -l # You should see: sndrpigooglevoicehat registered as Card 0 / Device 0
```

Step 3: Setup Virtual Environment & Install pinned Dependencies

```
Configure virtual environment and downgrade numpy to avoid tflite-runtime conflict: python3 -m venv venv source venv/bin/activate pip install -r requirements_pi.txt pip install "numpy<2"
```

Step 4: Run Real-time Listening Client

```
Run the listening script: python3 live_kws.py
```

Step 5: Run Resource Usage Audit

```
Run the measurement script in a separate terminal to log CPU and RAM footprint: python3 gate6_measure.py
```

7. Disclosures & Generalization Caveats

• **Data License**: The background noise subset incorporates sounds from the ESC-50 dataset, which is distributed under the **Creative Commons Attribution-NonCommercial (CC BY-NC 3.0)** license. This license must be disclosed if the system is presented in hackathons or public open-source releases.

• **Speaker Count Limitation**: Although splits are 100% speaker-disjoint, the test dataset contains only one unique speaker (Speaker 10) and validation contains two (Speakers 8 and 9). Before deploying in commercial or wide-audience environments, field testing should be conducted with a wider demographic of accents, vocal registers, and speech speeds.